

Vector applications in C++

list all vector applications with cpp

Got it 👍 — you want a **list of applications (use cases) of `std::vector` in C++**.

Here's a detailed breakdown:

◆ General Applications of `std::vector`

1. Dynamic Arrays

- Replace static arrays when size is unknown at compile time.
- Example: Storing user inputs of unknown length.

2. Stacks and Queues

- Implement stack (`push_back` , `pop_back`) and queue (`erase(begin())`) behavior.

3. Adjacency List in Graphs

- Each node stores neighbors in a `vector<int>`.
- Used in BFS, DFS, shortest path algorithms, etc.

4. Matrix Representation

- 2D vectors (`vector<vector<int>>`) for grids, DP tables, image processing, etc.

5. Dynamic Strings / Tokens

- Store tokens, substrings, or characters dynamically.

6. Sorting and Searching

- `sort(vec.begin(), vec.end())`
- `binary_search(vec.begin(), vec.end(), value)`

7. Frequency Counting / Histograms

- Vector used for counting occurrences (like hash tables for small key ranges).

8. Simulation of Data Buffers

- Store real-time incoming data (audio samples, sensor data, etc.).

9. Implementing Heaps / Priority Queues

- Vectors can be used with `make_heap` , `push_heap` , `pop_heap` .

10. Storing Database-like Records

- Each row/record is stored as a struct inside a vector.

11. Dynamic Polynomials

- Store coefficients of polynomial equations.

12. Game Development

- Store game objects (players, bullets, enemies, etc.).

13. Geometric Computations

- Store points, vectors, or 3D models dynamically.

14. File Handling

- Load entire files into a vector of chars or strings.

15. Sparse Arrays / Compressed Storage

- When using indices as positions and skipping unused ones.

16. Backtracking Problems

- Store partial solutions dynamically (e.g., N-Queens, Sudoku).

17. Dynamic Programming (DP) Tables

- Memoization and tabulation stored in vectors.

18. Implementing Undo/Redo Systems

- Store previous states in a vector and navigate with an index.

19. Scientific Computing

- Store experimental results, simulation outputs.

20. Event Handling Systems

- Store event listeners or actions dynamically.

♦ Small C++ Examples of Usage

cpp

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    // 1. Dynamic Array
    vector<int> arr = {1, 2, 3};
    arr.push_back(4);

    // 2. Stack
    vector<int> stack;
    stack.push_back(10);
    stack.pop_back();

    // 3. Graph Adjacency List
    int n = 3;
```

```

vector<vector<int>> graph(n);
graph[0].push_back(1);
graph[1].push_back(2);

// 4. DP Table
vector<vector<int>> dp(5, vector<int>(5, -1));

// 5. Sorting
sort(arr.begin(), arr.end());

return 0;
}

```

⚡ In short:

`std::vector` is one of the **most versatile containers** in C++ — useful anywhere you need **dynamic storage, random access, resizable arrays, or linear collections**.

👉 Do you want me to create a **categorized chart** (like **Algorithms, Data Structures, Real-world Systems**) of vector applications so it's easier to memorize for exams?

can I input values in vector using cin? show an example

Yes ☒ — you can take input into a `std::vector` using `cin`.

Here are a few common ways:

♦ Example 1: Input when size is known

```

cpp

#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
}

```

```

vector<int> v(n); // create vector of size n

cout << "Enter " << n << " numbers:\n";
for (int i = 0; i < n; i++) {
    cin >> v[i]; // direct indexing
}

cout << "You entered: ";
for (int x : v) cout << x << " ";
}

```

◆ Example 2: Input when size is unknown (use `push_back`)

```

cpp

#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v;
    int x;

    cout << "Enter numbers (end with -1):\n";
    while (cin >> x && x != -1) {
        v.push_back(x); // expand dynamically
    }

    cout << "You entered: ";
    for (int x : v) cout << x << " ";
}

```

◆ Example 3: Input all values in one line

```

cpp

#include <iostream>
#include <vector>

```

```
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> v;
    v.reserve(n); // optional, improves performance

    for (int i = 0; i < n; i++) {
        int val;
        cin >> val;
        v.push_back(val);
    }

    for (int x : v) cout << x << " ";
}
```

👉 So yes, you can use `cin` with vectors just like arrays, either by:

- **direct indexing** (`v[i]` when size is known), or
- **push_back** (when you don't know size beforehand).

Do you want me to also show you how to **input a 2D vector (matrix)** using `cin`?

what is "v.reserve(n);"

Great question 👍

`v.reserve(n);` is a **capacity reservation function** in C++ vectors.

◆ Meaning:

- `reserve(n)` tells the vector:
"I might store up to `n` elements, so please allocate memory for at least `n` slots in advance."
- It does **not** change the size of the vector, only its *capacity*.

◆ Difference Between `resize()` and `reserve()`

- `resize(n)` → Actually creates `n` elements (changes vector size).
 - `reserve(n)` → Only allocates memory for `n` elements (size remains the same).
-

◆ Example

cpp

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v;

    v.reserve(5); // allocates space for 5 elements
    cout << "Size: " << v.size() << ", Capacity: " << v.capacity() << endl;

    v.push_back(10);
    v.push_back(20);

    cout << "Size: " << v.size() << ", Capacity: " << v.capacity() << endl;

    return 0;
}
```

Output (example):

yaml

```
Size: 0, Capacity: 5
Size: 2, Capacity: 5
```

◆ Why use `reserve()` ?

- Without `reserve` , when a vector grows beyond its capacity, it reallocates memory (copies all old elements to new space).
- With `reserve` , you can **reduce reallocations** if you know the approximate number of elements in advance → faster performance.

⚡ In short:

`v.reserve(n);` pre-allocates memory for `n` elements to make future insertions more efficient.

👉 Do you want me to also show you an example where **not using** `reserve()` **makes insertion slower** compared to using it?

what is 'Dynamic Programming'? Explain using vector examples.

Nice one 👍 this ties everything together!

Let's break **Dynamic Programming (DP)** into a simple explanation first, then connect it with `vector` in C++.

♦ What is Dynamic Programming?

Dynamic Programming is a **problem-solving technique** where we break a problem into **smaller overlapping subproblems**, solve each subproblem **once**, and store the results to avoid recalculating.

- It's useful when problems have:
 1. **Overlapping Subproblems** → same subproblem appears multiple times.
 2. **Optimal Substructure** → the final solution can be built from solutions of subproblems.

👉 Instead of recursion doing the same calculation again and again, DP stores results (usually in an array or vector).

♦ Example 1: Fibonacci Numbers with Vector (Bottom-Up DP)

Problem: Find the n th Fibonacci number ($\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$).

C++ DP with `vector`:

```
cpp
```

```

#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cout << "Enter n: ";
    cin >> n;

    vector<long long> dp(n+1, 0); // DP array (vector)

    // Base cases
    dp[0] = 0;
    dp[1] = 1;

    // Build from bottom up
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }

    cout << "Fibonacci(" << n << ") = " << dp[n] << endl;
}

```

👉 Here, `vector dp` stores all intermediate results.

- No repeated calculations.
- Time: $O(n)$ instead of $O(2^n)$ (recursive).

◆ Example 2: Coin Change (Minimum Coins)

Problem: You have coins of certain values and want to make a sum `s` with the minimum number of coins.

C++ DP with `vector` :

```

cpp

#include <iostream>
#include <vector>
#include <climits>
using namespace std;

```



```

int main() {
    int S = 11;
    vector<int> coins = {1, 5, 7};

    vector<int> dp(S+1, INT_MAX);
    dp[0] = 0; // Base case: 0 coins needed for sum 0

    for (int i = 1; i <= S; i++) {
        for (int c : coins) {
            if (i - c >= 0 && dp[i - c] != INT_MAX) {
                dp[i] = min(dp[i], dp[i - c] + 1);
            }
        }
    }

    cout << "Minimum coins to make sum " << S << " = " << dp[S] << endl;
}

```

👉 `vector dp` stores the answer for every sub-sum.

- Each `dp[i]` = minimum coins to make sum `i`.

♦ Why Vectors Are Perfect for DP?

1. **Resizable** → we don't always know the exact state count.
2. **Random access** → constant-time access to subproblem results.
3. **Multi-dimensional** → `vector<vector<int>>` is perfect for 2D DP (like grid problems, LCS, knapsack).

⚡ In short:

Dynamic Programming = Smart recursion + memory.

Vectors in C++ act as the "memory table" to store and reuse results.

Would you like me to also show a **2D DP example** (like *Longest Common Subsequence* or *Knapsack*) using `vector<vector<int>>`? That's where vectors really shine.

Perfect 👍 Let's go step by step with **Knapsack Problem** (a very classic DP application).

♦ 0/1 Knapsack Problem (Definition)

You are given:

- `n` items, each with a **weight** `w[i]` and a **value** `v[i]`.
- A knapsack (bag) with maximum weight capacity `w`.

Goal:

Choose a subset of items such that:

- Total weight $\leq w$
- Total value is maximized

⚡ Each item can be taken **0 or 1 time only** → hence the name **0/1 Knapsack**.

♦ DP Approach

We use a **2D DP table**:

- `dp[i][w]` = maximum value achievable using the first `i` items with knapsack capacity `w`.

Transition formula:

```
lua

if (w[i] <= capacity):
    dp[i][w] = max(dp[i-1][w], dp[i-1][w - w[i]] + v[i])
else:
    dp[i][w] = dp[i-1][w]
```

♦ C++ Example Using `vector<vector<int>>`

```
cpp
```

```

#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n = 4; // number of items
    int W = 5; // max capacity of knapsack

    vector<int> weights = {2, 1, 3, 2};
    vector<int> values = {12, 10, 20, 15};

    // DP table: (n+1) x (W+1)
    vector<vector<int>> dp(n+1, vector<int>(W+1, 0));

    // Build DP table
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (weights[i-1] <= w) {
                dp[i][w] = max(dp[i-1][w], dp[i-1][w - weights[i-1]] + values[i-1]);
            } else {
                dp[i][w] = dp[i-1][w];
            }
        }
    }

    cout << "Maximum value in Knapsack = " << dp[n][W] << endl;

    return 0;
}

```

◆ Example Walkthrough

Items:

- Item1: weight=2, value=12
- Item2: weight=1, value=10
- Item3: weight=3, value=20
- Item4: weight=2, value=15

Capacity `w=5` .

👉 Optimal choice: Item2 + Item3 → weight=4, value=30.

So the program outputs:

```
java
```

```
Maximum value in Knapsack = 37
```

(because actually the best is Item1 + Item2 + Item4 = 2+1+2=5, value=12+10+15=37).

♦ Why `vector` is useful here?

- `vector<vector<int>> dp(n+1, vector<int>(W+1, 0));` → easily creates a **2D DP table**.
- Automatic memory management, dynamic resizing.
- Easy to index like an array.

⚡ That's the **classic DP + vector** solution for 0/1 Knapsack!

👉 Do you also want me to show the **space-optimized version** (using just a 1D `vector<int>` instead of 2D)?

what is "Fractional Knapsack"?

Good question 👍 this is the **other famous version** of the Knapsack problem.

♦ Fractional Knapsack Problem (Definition)

- You are given `n` items, each with a **weight** `w[i]` and a **value** `v[i]`.
- Knapsack (bag) has maximum capacity `w`.
- **Unlike 0/1 Knapsack**, here **you can take fractions of items** (i.e., split an item into smaller parts).

Goal:

Maximize the **total value** of items in the knapsack, without exceeding the capacity.

◆ Key Idea

Since we can take fractions, the **Greedy approach** works:

1. Compute **value/weight ratio** for each item.
2. Sort items in **descending order of ratio**.
3. Pick items fully until the knapsack is full.
4. If the next item can't fully fit, take the fraction that fits.

👉 Unlike **0/1 Knapsack (DP)**, this problem is solved with **Greedy Algorithm** in $O(n \log n)$.

◆ Example

Items:

- Item1: weight=10, value=60 → ratio = 6
- Item2: weight=20, value=100 → ratio = 5
- Item3: weight=30, value=120 → ratio = 4

Capacity = $W = 50$.

Steps:

- Take Item1 fully → weight=10, value=60
- Take Item2 fully → weight=30, value=100 (total so far = 160)
- Item3: only 20/30 fits → value added = $20 * 4 = 80$

✅ Maximum value = $60 + 100 + 80 = 240$.

◆ C++ Example with `vector`

cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Item {
    int value, weight;
};
```

```

bool cmp(Item a, Item b) {
    double r1 = (double)a.value / a.weight;
    double r2 = (double)b.value / b.weight;
    return r1 > r2; // sort by value/weight ratio
}

double fractionalKnapsack(int W, vector<Item>& items) {
    sort(items.begin(), items.end(), cmp);

    int curWeight = 0;
    double finalValue = 0.0;

    for (auto& it : items) {
        if (curWeight + it.weight <= W) {
            // take whole item
            curWeight += it.weight;
            finalValue += it.value;
        } else {
            // take fraction
            int remain = W - curWeight;
            finalValue += it.value * ((double)remain / it.weight);
            break; // knapsack is full
        }
    }
    return finalValue;
}

int main() {
    int W = 50;
    vector<Item> items = {{60, 10}, {100, 20}, {120, 30}};

    cout << "Maximum value in Knapsack = "
         << fractionalKnapsack(W, items) << endl;

    return 0;
}

```

Output:

```
java
```

Maximum value in Knapsack = 240

◆ Difference from 0/1 Knapsack

Feature	0/1 Knapsack	Fractional Knapsack
Take items?	Whole item or none (0/1)	Can take fractions
Technique	Dynamic Programming	Greedy (sort ratios)
Time Complexity	$O(n * W)$	$O(n \log n)$
Type of Problem	NP-Hard (exact solution)	Greedy works optimal

⚡ In short:

- **0/1 Knapsack** → Use **DP**.
- **Fractional Knapsack** → Use **Greedy**.

👉 Want me to also show you a **side-by-side code comparison** of 0/1 vs Fractional Knapsack so you can clearly see the difference?

show a simple queue using c++

Got it 👍 Let's build a **simple queue in C++**.

We'll do it two ways: using **STL queue** (easy) and using a **vector/array manually** (to understand).

◆ 1. Queue using STL (`<queue>`)

The Standard Template Library (STL) provides a ready-made `queue` container.

```
cpp
```

```

#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;

    // Enqueue (push elements)
    q.push(10);
    q.push(20);
    q.push(30);

    cout << "Front: " << q.front() << endl; // 10
    cout << "Back: " << q.back() << endl; // 30

    // Dequeue (remove elements)
    q.pop(); // removes 10

    cout << "Front after pop: " << q.front() << endl; // 20

    // Traverse queue
    cout << "Queue elements: ";
    while (!q.empty()) {
        cout << q.front() << " ";
        q.pop();
    }
}

```

Output:

yaml

```

Front: 10
Back: 30
Front after pop: 20
Queue elements: 20 30

```

◆ 2. Queue Implementation Using `vector` (Manual)

If you want to understand the working:

cpp

```
#include <iostream>
#include <vector>
using namespace std;

class Queue {
    vector<int> data;
    int frontIndex = 0; // track front

public:
    void enqueue(int x) {
        data.push_back(x);
    }

    void dequeue() {
        if (!empty()) frontIndex++;
    }

    int front() {
        if (!empty()) return data[frontIndex];
        throw out_of_range("Queue is empty");
    }

    bool empty() {
        return frontIndex >= data.size();
    }
};

int main() {
    Queue q;
    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    cout << "Front: " << q.front() << endl; // 10
    q.dequeue();

    cout << "Front after dequeue: " << q.front() << endl; // 20
}
```

◆ Queue Operations

- **enqueue(x)** → Add element at the end.
 - **dequeue()** → Remove element from the front.
 - **front()** → Access first element.
 - **empty()** → Check if queue is empty.
-

⚡ In short:

- Use `std::queue` for real-world coding (easy & efficient).
- Implement manually with `vector / array` if you want to **learn internals**.

👉 Do you want me to also show you how to make a **circular queue** (fixed-size, reuses space)? That's usually the next step after this.

explain:

```
void dequeue() {  
    if (!empty()) frontIndex++;  
}
```

why did we use `!empty()` instead of `empty()`?

Ah, good observation! Let's carefully break it down.

Here's the function you asked about:

cpp

```
void dequeue() {  
    if (!empty()) frontIndex++;  
}
```

◆ What `empty()` Does

In our `Queue` class:

cpp

```
bool empty() {  
    return frontIndex >= data.size();  
}
```

- `empty()` returns `true` if the queue has no elements left.
 - `empty()` returns `false` if there are still elements.
-

♦ Why `!empty()` ?

We only want to **increment** `frontIndex` (i.e., remove the front element) if the queue is **not empty**.

- `!empty()` → “queue is **not empty**”
- `empty()` → “queue is **empty**”

If we wrote:

cpp

```
if (empty()) frontIndex++;
```

It would try to remove an element **even when the queue is empty**, which is wrong — could lead to `frontIndex` going out of bounds.

♦ Analogy

Think of a real-life queue (like at a ticket counter):

- You **cannot remove a person from the front if the line is empty**.
 - `!empty()` = “there’s someone in line, okay to remove”
 - `empty()` = “line is empty, don’t remove”
-

✓ So the `!empty()` check is **safety measure** to make sure we don’t remove elements from an empty queue.

If you want, I can also **explain why this simple vector-based queue is not fully efficient** and how **circular queue** fixes it. It's a common follow-up question.